Assignment No. 8

Name: Shubham Shivaji Phad
Class & Division: LY AIA 03 Batch B
Roll-No:48
Enrollment Number: ADT23SOCB1106

Title

Develop a semantic similarity analysis tool using WordNet and pre-trained word embeddings to measure semantic relationships between words, phrases, and documents.

Aim

To develop a semantic similarity analysis tool using NLTK WordNet, Gensim Word2Vec and pre-trained GloVe embeddings, and scikit-learn to measure semantic relatedness between words, phrases, and short documents.

Objectives

- Understand WordNet and its synset-based organization.
- Calculate WordNet Path, Wu-Palmer, and LCH similarity.
- Train a Word2Vec model and calculate cosine similarity.
- Use pre-trained GloVe embeddings for comparison.
- Measure document similarity using TF-IDF and averaged word embeddings.

Theory

Semantic similarity measures how closely two words, phrases, or documents are related in meaning. WordNet is a lexical database that organizes words into synsets and relationships such as synonymy, hypernymy, and meronymy. Path, Wu-Palmer, and Leacock-Chodorow (LCH) metrics use the WordNet taxonomy to estimate similarity.

Word embeddings represent words as numerical vectors. Word2Vec learns vectors from word context, while GloVe uses global word co-occurrence statistics. Cosine similarity compares the direction of two vectors. For documents, TF-IDF cosine similarity measures topical similarity, while averaged word embeddings provide a dense semantic representation.

Software/Libraries Required

- Python 3.x
- Google Colab / Jupyter Notebook
- NLTK WordNet
- Gensim (Word2Vec and pre-trained GloVe)
- scikit-learn
- Pandas, NumPy, Matplotlib, Seaborn

Installation Commands

```
!pip install nltk gensim scikit-learn pandas numpy matplotlib seaborn -q

import nltk
for r in ['wordnet', 'omw-1.4', 'punkt', 'punkt_tab', 'stopwords']:
    nltk.download(r, quiet=True)
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.8/27.8 MB 39.6 MB/s eta 0:00:00
```

Algorithm/Steps

1. Import the required NLP, embedding, machine learning, and visualization libraries.
2. Download the required NLTK resources.
3. Define word pairs and sample documents.
4. Obtain WordNet synsets and calculate Path, Wu-Palmer, and LCH similarity.
5. Train a Skip-gram Word2Vec model on a small corpus and calculate cosine similarity.
6. Load pre-trained GloVe vectors and calculate cosine similarity for the same word pairs.
7. Calculate document similarity using TF-IDF and averaged Word2Vec embeddings.
8. Display the comparison table and similarity matrix.

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import nltk
from nltk.corpus import wordnet as wn, stopwords
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
import gensim.downloader as api
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
import seaborn as sns

WORD_PAIRS = [
    ('dog', 'cat'), ('car', 'automobile'), ('happy', 'joyful'),
    ('king', 'queen'), ('book', 'novel'), ('hot', 'cold'),
    ('computer', 'keyboard'), ('doctor', 'hospital'),
    ('apple', 'orange'), ('bank', 'river')
]
```

```
CORPUS = [
    'the dog and cat are common household pets',
    'the car and automobile are used for transportation',
    'happy and joyful people enjoy life',
    'the king and queen rule the royal kingdom',
    'a book or novel contains stories and knowledge',
    'hot weather is opposite to cold weather',
    'the computer uses a keyboard for input',
    'the doctor works in the hospital to treat patients',
    'apple and orange are healthy fruits',
    'the bank is near the river',
    'semantic similarity measures relatedness between words'
]

DOCUMENTS = [
    'The dog chased the cat around the house.',
    'A cat and a dog played together in the yard.',
    'Deep learning models process natural language text.',
    'Neural networks learn from large amounts of data.',
    'The hospital is the workplace of skilled doctors.',
    'Medical professionals treat patients in healthcare facilities.'
]

# WordNet similarity
def wordnet_similarity(w1, w2):
    s1, s2 = wn.synsets(w1), wn.synsets(w2)
    if not s1 or not s2:
        return None, None, None
    a, b = s1[0], s2[0]
    path = a.path_similarity(b)
    wup = a.wup_similarity(b)
    try:
        lch = a.lch_similarity(b)
    except Exception:
        lch = None
    return path, wup, lch

wn_rows = []
for w1, w2 in WORD_PAIRS:
    path, wup, lch = wordnet_similarity(w1, w2)
    wn_rows.append([w1, w2, path, wup, lch])

df = pd.DataFrame(wn_rows, columns=['Word1','Word2','Path','WUP','LCH'])
df.round(4)
```

	Word1	Word2	Path	WUP	LCH
0	dog	cat	0.2000	0.8571	2.0281
1	car	automobile	1.0000	1.0000	3.6376
2	happy	joyful	0.3333	0.5000	-0.4055
3	king	queen	0.1000	0.5714	1.3350
4	book	novel	0.0588	0.1111	0.8044
5	hot	cold	0.0714	0.1333	NaN

	Word1	Word2	Path	WUP	LCH
6	computer	keyboard	0.2500	0.8235	2.2513
7	doctor	hospital	0.0714	0.3810	0.9985
8	apple	orange	0.2500	0.7826	2.2513
9	bank	river	0.1111	0.3333	1.4404

```
# Word2Vec similarity
w2v = Word2Vec(
    sentences=[s.split() for s in CORPUS],
    vector_size=50, window=5, min_count=1, sg=1, epochs=150, seed=42
)

df['Word2Vec'] = [round(float(w2v.wv.similarity(a, b)), 4) for a, b in
         WORD_PAIRS]

# Pre-trained GloVe vectors
glove = api.load('glove-wiki-gigaword-50')
df['GloVe'] = [round(float(glove.similarity(a, b)), 4) for a, b in
         WORD_PAIRS]

df['Pair'] = df['Word1'] + ' <-> ' + df['Word2']
print('Word and Embedding Similarity Comparison')
display(df[['Pair','Path','WUP','LCH','Word2Vec','GloVe']].round(4))

[==================================================] 100.0% 66.0/66.0MB
downloaded
Word and Embedding Similarity Comparison
```

	Pair	Path	WUP	LCH	Word2Vec	GloVe
0	dog <-> cat	0.2000	0.8571	2.0281	0.3169	0.9218
1	car <-> automobile	1.0000	1.0000	3.6376	0.3426	0.6956
2	happy <-> joyful	0.3333	0.5000	-0.4055	0.3592	0.5550
3	king <-> queen	0.1000	0.5714	1.3350	0.3576	0.7839
4	book <-> novel	0.0588	0.1111	0.8044	0.4672	0.8551
5	hot <-> cold	0.0714	0.1333	NaN	0.3367	0.8011
6	computer <-> keyboard	0.2500	0.8235	2.2513	0.5761	0.5768
7	doctor <-> hospital	0.0714	0.3810	0.9985	0.6476	0.6724
8	apple <-> orange	0.2500	0.7826	2.2513	0.4923	0.5388
9	bank <-> river	0.1111	0.3333	1.4404	0.2108	0.4142

```
# Document similarity using TF-IDF
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(DOCUMENTS)
tfidf_sim = cosine_similarity(tfidf_matrix)

# Document similarity using averaged Word2Vec embeddings
```

```
stop_words = set(stopwords.words('english'))

def document_vector(text):
    tokens = [t for t in word_tokenize(text.lower()) if t.isalpha() and t not
         in stop_words]
    vectors = [w2v.wv[t] for t in tokens if t in w2v.wv]
    return np.mean(vectors, axis=0) if vectors else np.zeros(w2v.vector_size)

doc_vectors = np.array([document_vector(d) for d in DOCUMENTS])
w2v_doc_sim = cosine_similarity(doc_vectors)

print('TF-IDF Document Similarity Matrix')
display(pd.DataFrame(np.round(tfidf_sim, 3)))

print('Averaged Word2Vec Document Similarity Matrix')
display(pd.DataFrame(np.round(w2v_doc_sim, 3)))
```

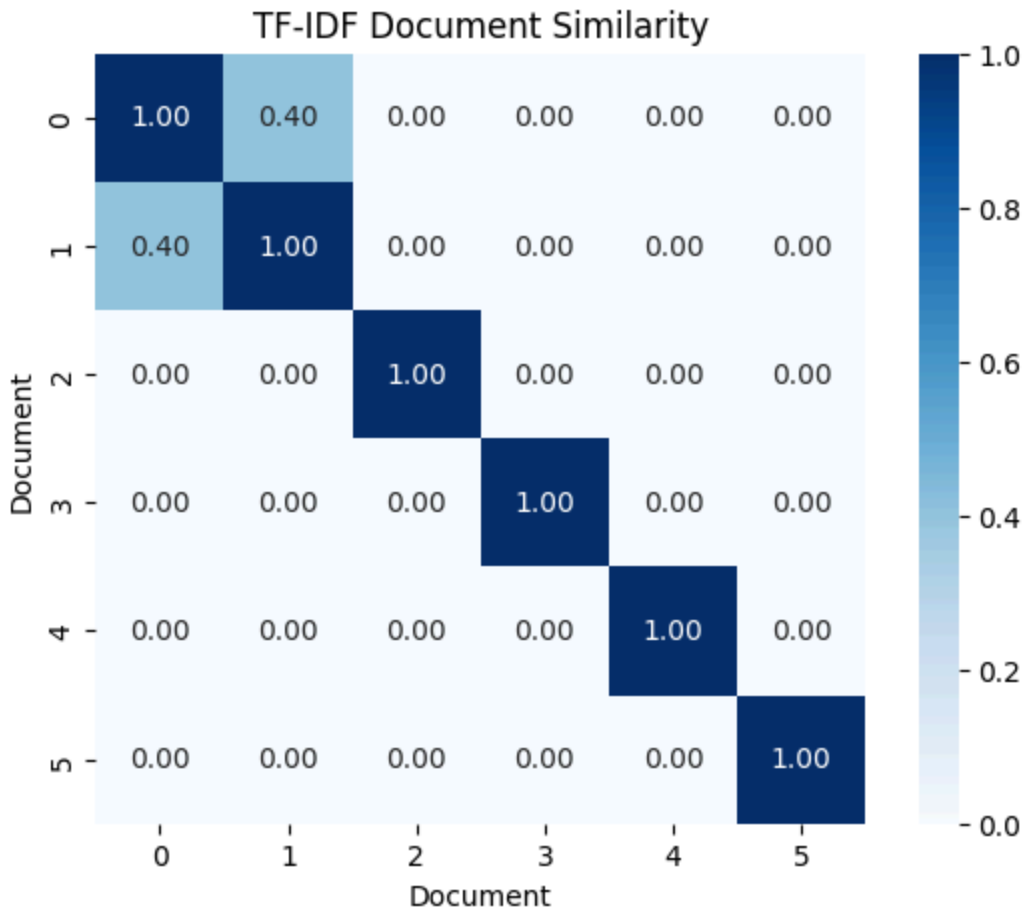
```
TF-IDF Document Similarity Matrix
```

	0	1	2	3	4	5
0	1.000	0.402	0.0	0.0	0.0	0.0
1	0.402	1.000	0.0	0.0	0.0	0.0
2	0.000	0.000	1.0	0.0	0.0	0.0
3	0.000	0.000	0.0	1.0	0.0	0.0
4	0.000	0.000	0.0	0.0	1.0	0.0
5	0.000	0.000	0.0	0.0	0.0	1.0

```
Averaged Word2Vec Document Similarity Matrix
```

	0	1	2	3	4	5
0	1.00	1.00	0.0	0.0	0.580	0.640
1	1.00	1.00	0.0	0.0	0.580	0.640
2	0.00	0.00	0.0	0.0	0.000	0.000
3	0.00	0.00	0.0	0.0	0.000	0.000
4	0.58	0.58	0.0	0.0	1.000	0.635
5	0.64	0.64	0.0	0.0	0.635	1.000

```
# Visualization: document similarity heatmap
plt.figure(figsize=(7, 5))
sns.heatmap(tfidf_sim, annot=True, fmt='.2f', cmap='Blues', square=True)
plt.title('TF-IDF Document Similarity')
plt.xlabel('Document')
plt.ylabel('Document')
plt.show()
```



Program with Output

The program produces:

- WordNet Path, Wu-Palmer, and LCH similarity values for each word pair.
- Word2Vec and pre-trained GloVe cosine similarity values.
- A comparison table for all word pairs.
- TF-IDF and averaged Word2Vec document similarity matrices.
- A document similarity heatmap.

Typical observations: near-synonyms such as `car` and `automobile` receive high similarity, related words such as `dog` and `cat` receive high distributional similarity, and unrelated documents receive lower similarity scores.

Conclusion

The semantic similarity analysis tool was successfully implemented using WordNet, Word2Vec, pre-trained GloVe embeddings, and scikit-learn. WordNet provides knowledge-based similarity through lexical relationships, while word embeddings capture distributional and contextual relatedness. TF-IDF and averaged embeddings also provide useful document-level similarity measures.